📘 Setup Guide

Prisma + Neon Setup Guide

A complete step-by-step guide to setting up Prisma ORM with Neon serverless PostgreSQL — including the new Prisma v7 driver adapter pattern.

Prisma v7 | Neon Postgres | TypeScript | Bun / Node.js | Driver Adapters

1

What is Prisma & Neon?

TOOL	WHAT IT IS	WHY USE IT
Prisma	Next-gen TypeScript ORM	Type-safe DB queries, auto-migrations, schema-first
Neon	Serverless PostgreSQL	Auto-scaling, branching, free tier, instant setup

💡 Neon gives you a free hosted Postgres database at **neon.tech**. No local DB setup needed!

2

Step 1 — Create a Neon Database

1 **Go to neon.tech**

Sign up for a free account at https://neon.tech

2 **Create a New Project**

Click "New Project" → give it a name → select a region closest to you

3 **Copy the Connection String**

Go to Dashboard → Connection Details → copy the `DATABASE_URL`

4 **Save it in .env**

Create a `.env` file in your server root and paste the URL

.ENV

```
DATABASE_URL="postgresql://username:password@ep-xxx.us-east-2.aws.neon.
```

⚠ Add **.env** to your **.gitignore** — never commit your database URL!

3

Step 2 — Install Dependencies

USING BUN

TERMINAL

```
bun add @prisma/client @prisma/adapter-pg pg
bun add -d prisma @types/pg
```

USING NPM

TERMINAL

```
npm install @prisma/client @prisma/adapter-pg pg
npm install -D prisma @types/pg
```

PACKAGE	PURPOSE
`prisma`	CLI tool for migrations & schema management (devDependency)
`@prisma/client`	The generated type-safe database client
`@prisma/adapter-pg`	Prisma v7 driver adapter for PostgreSQL
`pg`	Native PostgreSQL driver for Node.js

4

Step 3 — Initialize Prisma

TERMINAL

```
bunx prisma init
# or
npx prisma init
```

This creates two files:

FILE	PURPOSE
`prisma/schema.prisma`	Your database schema (models, relations)
`prisma.config.ts`	Prisma v7 config file (connection URL goes here)

5

Step 4 — Configure schema.prisma

Prisma v7 Breaking Change: The `url` field is no longer allowed in `schema.prisma`. It now lives in `prisma.config.ts`.

PRISMA/SCHEMA.PRISMA

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  // ✅ No url field here in Prisma v7!
}

model User {
  id        String   @id
  email     String   @unique
  name      String?
  createdAt DateTime @default(now())
}
```

6

Step 5 — Configure prisma.config.ts

PRISMA.CONFIG.TS

```
import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
  },
  datasource: {
    url: process.env.DATABASE_URL,
  },
});
```

This is the **Prisma v7 way** — the DATABASE_URL is read from your `.env` file via dotenv and passed here.

7

Step 6 — Create the DB Connection

In Prisma v7, you must use the **driver adapter pattern** to instantiate PrismaClient:

DB/CONNECTION.TS

```
import { Pool } from 'pg';
import { PrismaPg } from '@prisma/adapter-pg';
import { PrismaClient } from '@prisma/client';

// 1. Create a connection pool
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

// 2. Create the Prisma adapter
const adapter = new PrismaPg(pool);

// 3. Instantiate PrismaClient with the adapter
export const prisma = new PrismaClient({ adapter });
```

⚠ Do **NOT** use `new PrismaClient()` without the adapter in Prisma v7 with pg — it will error.

8

Step 7 — Generate & Migrate

GENERATE THE PRISMA CLIENT

TERMINAL

```
bunx prisma generate
```

CREATE YOUR FIRST MIGRATION

TERMINAL

```
bunx prisma migrate dev --name init
```

PUSH SCHEMA WITHOUT MIGRATION (QUICK DEV)

TERMINAL

```
bunx prisma db push
```

COMMAND	WHEN TO USE
`prisma generate`	After every schema change to regenerate the client
`prisma migrate dev`	Development — creates SQL migration files
`prisma db push`	Rapid prototyping — no migration files created
`prisma migrate deploy`	Production — applies pending migrations
`prisma studio`	Visual database browser in the browser

9

Step 8 — Use Prisma in Your Server

SERVER.TS

```
import express from 'express';
import 'dotenv/config';
import { prisma } from './db/connection';

const app = express();
app.use(express.json());

// ✅ Test route — fetch all users
app.get('/users', async (req, res) => {
  const users = await prisma.user.findMany();
  res.json(users);
});

// ✅ Create a user
app.post('/users', async (req, res) => {
  const { id, email, name } = req.body;
  const user = await prisma.user.create({
    data: { id, email, name },
  });
  res.json(user);
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

10

Common Errors & Fixes

ERROR	CAUSE	FIX
`url is no longer supported in schema files`	Prisma v7 breaking change	Remove `url` from schema.prisma, use prisma.config.ts
`@prisma/client did not initialize`	Forgot to run generate	Run `bunx prisma generate`
`P1001: Can't reach database`	Wrong or missing DATABASE_URL	Check .env file and Neon connection string
`SSL required`	Neon requires SSL	Add `?sslmode=require` to connection string
`PrismaClient takes no arguments`	Old API usage	Pass `{ adapter }` to PrismaClient

11

Quick Reference Cheatsheet

PRISMA QUERIES

```
// Find all
const all = await prisma.user.findMany();

// Find one
const user = await prisma.user.findUnique({ where: { id: "123" } });

// Create
const created = await prisma.user.create({ data: { id, email, name } })

// Update
const updated = await prisma.user.update({
  where: { id: "123" },
  data: { name: "New Name" },
});

// Delete
await prisma.user.delete({ where: { id: "123" } });

// With relations
const userWithChats = await prisma.user.findUnique({
  where: { id: "123" },
  include: { chats: true },
});
```